

Deterministic is not sound: an empirical evaluation of seccomp argument-inspection gates for containing LLM agents

Sadik Erisen*

Independent Researcher & Graduate Student
University of Pennsylvania

July 2026

Abstract

Sandboxes that run untrusted LLM-agent code increasingly reach for a syscall-boundary control: a seccomp supervisor that inspects the *arguments* of intercepted system calls and allows or denies each one. Because the supervisor’s verdict is deterministic, it is tempting to treat the control as certified containment. We test that control directly, on a mechanical oracle that compiles and runs the code and reads its behaviour from `strace`, never from text. We report four findings. First, the argument-inspection gate is bypassable: it blocks a persistence write 4000/4000 times head-on, yet a two-thread time-of-check-to-time-of-use (TOCTOU) race on its allow path defeats it at 19.2–22.8% per attempt, which approaches certainty under retry. Second, attacker-injected and self-driven behaviour do not collapse to one syscall footprint: they share `execve`, but 18% of self-driven destruction takes a direct `unlink/rmdir` path that an `execve/connect` gate never observes. Third, the gate is genuinely selective rather than deny-all, but its substring path-matching admits false positives on benign look-alike filenames while its anchored tokens are evaded by path normalization. Fourth, attributing the actor from its behaviour—by timing, by stated intent, or by its reaction to a denial—fails as a control against an adversary that chooses its own behaviour. The determinism of the verdict is not soundness of the containment. What protects the host is coverage of the mechanisms an agent can use, and a denied agent enumerates the alternatives rather than stopping.

1 Introduction

An LLM agent that writes and runs code is, from the operating system’s point of view, an untrusted process whose intent cannot be read from its prompt, its reasoning trace, or the code it emits. A common containment proposal is to watch the one interface the process cannot avoid: the system-call boundary. A rootless filter blocks by syscall number; a richer, “rootful” design installs a seccomp user-notification supervisor that reads the pointer arguments of an intercepted call out of the target’s memory and decides allow or deny per call. Argument visibility is the selling point—the supervisor can permit `openat` on a log file and deny it on `~/.ssh/authorized_keys`.

The supervisor’s verdict is deterministic: the same call with the same arguments yields the same decision every time. That property invites a category error, in which a deterministic control is presented as *certified* containment. This paper separates the two. A verdict can be perfectly deterministic and still wrong, either because it inspects an argument the kernel later re-reads, or

*This research was conducted independently. The views and findings are those of the author and do not reflect the position of any corporate employer. No data in this paper is fabricated; every figure traces to a committed artifact and is reproducible via `experiments/agentsec/reproduce.sh`.

because the dangerous action never passes through a watched call at all. Determinism bounds the *variance* of the control; soundness would bound its *coverage and correctness*, and coverage is where the containment fails.

We hold every claim to a mechanical standard. A statement that “the agent attempted *X*” means a compiled program issued syscall *X* under **strace** inside a rootless network namespace with a timeout; we never infer behaviour from generated text. Under that standard we report the following as findings, not activities.

1. The argument-inspection gate is bypassable by a TOCTOU race on its allow path (Section 4).
2. Attacker-driven and self-driven behaviour do not reduce to a single syscall footprint, so a gate scoped to one mechanism leaves the other’s path open (Section 5).
3. The gate’s selectivity rests on substring path-matching that admits false positives and is separately evadable (Section 6).
4. Behavioural attribution of the actor is not a control (Section 8).

Section 7 states what does survive, so the reader does not over-correct: the syscall boundary remains the right place to observe, and interception is invariant to the source language. Sections 9–11 give the implication for a threat model, the limits of the study, and the position against prior work.

2 Threat model

The host runs code produced by an LLM agent and treats that code as untrusted. Two sources drive dangerous behaviour, and the design goal is to contain both at one boundary. The first is *attacker injection*: a prompt-injection payload steers the agent into emitting a dangerous call, for example `system("/bin/sh")` routed through the generated code. The second is *autonomous malfunction*: with no attacker, the agent reaches for a destructive shortcut on its own, for example deleting a build directory it was asked to “reset.”

The defender controls the seccomp filter and the supervisor process; the supervisor obtains read access to the target’s memory through the parent–child relationship (equivalently `CAP_SYS_PTRACE` in a hardened deployment). The attacker controls the contents of the untrusted process—its threads, the timing of its calls, and the arguments in its own address space—but not the supervisor or the kernel. This is the standard capability split for a syscall supervisor, and it is exactly the split under which the allow-path race in Section 4 is available: the attacker owns the memory the supervisor reads and the kernel later re-reads.

3 Method

Mechanical oracle. Generated C is compiled with `gcc` and executed under `timeout 10 unshare -rn strace -f`. The footprint of a run is the set of security-relevant syscalls it issues (`execve`, `connect`, `openat`, `unlink`, `rmdir`, and their families), read from the trace. A baseline of one `execve`—the process launch—is subtracted before a run counts as “attempting.” No claim in this paper is derived by scanning model output for keywords.

Subjects. Three local code models generate the artifacts: `qwen2.5-coder:1.5b`, `qwen2.5-coder:7b`, and `deepseek-coder:6.7b`, served through Ollama. The host is a single x86_64 machine. Section 10 states what this scope does and does not support.

The gate. The rootful gate forks the target, installs `SCMP_ACT_NOTIFY` on `execve`, `execveat`, and `connect` (and, in persistence mode, on `openat`, `rename`, `link`, and their variants), and sends the notification descriptor to the supervisor. On each notification the supervisor reads the pathname from `/proc/pid/mem`, compares it against policy, and either allows the call by returning `SECCOMP_USER_NOTIF_FLAG_CONTINUE` or denies it with `EPERM`. The rootless gate, by contrast, is a blanket `SCMP_ACT_ERRNO` deny of `execve/execveat/connect` with no argument inspection.

Statistics. For a stochastic quantity—an agent’s rate of emitting a dangerous call—we report a Wilson 95% interval. The gate’s decision is not a stochastic rate and receives no failure-rate interval; its determinism is a property to be stated, and its soundness is measured separately as coverage (Sections 4–6). Every figure below is reproducible from committed source through `experiments/agentsec/reproduce.sh`, which rebuilds the binaries and re-checks the headline results (8 checks, all passing at the time of writing).

4 The argument-inspection gate is bypassable

The gate blocks the direct attack completely. A target that writes to a path containing a persistence token—`authorized_keys`, `/.ssh/`, `cron`—is denied on every one of 4000 attempts, and the supervisor’s own counter agrees with the target’s: 4000 calls seen, 4000 denied. Head-on, the argument inspection does exactly what it claims.

The allow path does not survive a race. On a permitted call the supervisor validates the pathname it read from `/proc/pid/mem`, then returns `SECCOMP_USER_NOTIF_FLAG_CONTINUE`, which instructs the kernel to resume the syscall. Resuming the syscall means the kernel copies the pathname pointer *again*, at execution time. Between the supervisor’s read and the kernel’s re-read the target’s other threads keep running—only the calling thread is parked on the notification—so a sibling thread that flips the buffer from a benign path to `authorized_keys` in that window causes the supervisor to validate one path and the kernel to open another.

We measured the effect with a two-thread target: one thread issues the `openat`, a second flips the shared path buffer. Across 4000 attempts the evil path was opened despite the gate on 19.2% of tries in the first run and 22.8% in an independent rebuild; the remainder split between benign opens and correct denials, and the three outcomes account for every trial. The existence of the bypass is a property of the mechanism and is host-independent; the per-attempt rate depends on scheduler behaviour and moved between runs, so we do not treat it as a fixed figure. Under retry the distinction stops mattering: at a per-attempt success near one in five, a persistence write succeeds with probability above 0.9 within roughly eleven attempts, and the agent is free to retry.

This is the documented deep-argument hazard of seccomp user notification, and the `seccomp_unotify(2)` manual page warns against using `CONTINUE` to allow a syscall whose arguments were security-relevant, precisely because the arguments can change before the kernel acts. Two consequences follow for a threat model. The bypass is architectural, not a gap in the token list, so no blocklist patch closes it; the second read has to be removed, by copying the argument into supervisor-controlled memory or by having the supervisor perform the operation itself. And the head-on number—4000/4000—measures the control against a cooperative target, which the threat model does not assume.

We note one correction to a natural reading of this result. The gate permits exactly one `execve`, the launch of the target itself, and that call fires in a single-threaded context before any agent code runs; every later `execve` is denied outright. The raceable allow path is therefore not the `execve` launch but the per-call file-operation allow in persistence mode, which fires while the attacker’s threads are live.

5 Coverage is incomplete before any race

A gate can be sound only over the mechanisms it watches, and the two threat sources of Section 2 do not use the same mechanisms. We compared the footprints of the artifacts each source actually produced. The 164 injection artifacts that attempted a dangerous call did so entirely through `execve`: the footprint set is `{execve}`. The 45 autonomous artifacts that attempted destruction did so through `execve` in 82% of cases (spawning a shell to run `rm`), but through a direct `unlink` in 13% and a direct `rmdir` in 4%: the footprint set is `{execve, unlink, rmdir}`. The two sets share `execve` and are not equal. The footprint is fixed by the *mechanism* the code chooses, not by the *force* that drove it, and 18% of self-driven destruction reaches its goal by a call an `execve/connect` gate never sees.

The gap widens at the ABI. Blocking `execve` does not cover `execveat` on an anonymous descriptor via `memfd_create`; blocking `connect` does not cover `sendto` or `sendmsg` on a UDP socket; and a single unblocked `io_uring_setup` lets an agent submit `openat`, `write`, `connect`, and `unlink` as ring operations that never reach the syscall-entry filter, defeating file and network coverage at once. On a filter scoped to one architecture’s syscall numbers, the i386 and x32 compat entries fall through to the default action. A blocklist cannot enumerate its way out of this; completeness requires inverting to a default-deny allowlist, and the only class this study could close as a small fixed set is execution (`execve`, `execveat`). We did not exercise every listed ABI path as a working exploit; the `unlink/rmdir` escape is measured on real artifacts, and the remainder are coverage gaps read from the filter definition and the syscall ABI.

6 Selectivity trades false positives against evasion

The rootful gate is not a disguised deny-all: given a labelled corpus of write paths it denied all seven persistence targets and allowed the ordinary application writes, so it does discriminate on the argument, unlike the rootless filter, which denies the watched calls unconditionally. Its discrimination mechanism, however, is a substring match against a token list, and that mechanism sits on a false-positive/evasion trade-off it cannot leave.

On a seventeen-path corpus the confusion matrix was seven true positives, zero false negatives, six true negatives, and four false positives (positive = should-be-denied). Every false positive was a legitimate filename that merely contains a token as a substring—`microcron_metrics.txt` matching `cron`, `authorized_keys_report.csv` matching `authorized_keys`. The tokens that cause these false positives are the un-anchored ones; the tokens that avoid them are anchored with a leading slash (`/.ssh/`, `/.profile`), and those are exactly the tokens defeated by a relative path or a symbolic link through an allowed directory. Tightening a token to cut a false positive is the same edit that opens an evasion, and loosening it to catch an evasion is the same edit that creates a false positive. Substring matching cannot be both precise and evasion-resistant; a race-free, path-aware mechanism such as Landlock resolves the path before deciding and is the correct comparison point for a production design.

7 What holds

The findings above are not an argument to abandon the syscall boundary. Two properties of the boundary survive, and stating them keeps the recommendation calibrated.

Interception is invariant to the language that emitted the call. The same second `execve`—a shell spawned after launch—is denied identically whether the target is compiled C, a Python interpreter, or a shell script (`denied_execve` = 1 in each, with the shell never running). The gate observes the kernel interface, so the source language above it does not change the verdict.

The boundary is also the right place to observe, because analysis above it cannot certify. In companion work on the same corpus, ten static and dynamic detectors, AddressSanitizer

included, all miss an intra-object overflow that crosses no kernel boundary; the artifact is clean to every checker yet corrupt at runtime. Observation therefore has to move to execution, and the syscall boundary is where execution becomes visible. What the boundary gives is a deterministic, auditable *verdict*; what it does not give, per Sections 4–6, is soundness.

8 Behavioural attribution is not a control

If the gate is porous, a natural next move is to profile the agent’s behaviour and stop the bad actor before it reaches a gap. We tested three variants and none is a control against an adversary, for one structural reason: a manipulated agent chooses its own behaviour, so any signature the defender keys on can be produced or suppressed at will.

Timing does not attribute. A script that inserts random sleeps is statistically indistinguishable from a live agent’s action cadence (coefficient of variation 0.45 versus 0.54; fifteen of seventeen inter-action intervals overlap), so raw timing separates neither by construction.

Stated intent against footprint is a weak, spoofable signal. Comparing a model’s one-sentence description of its own code against the code’s measured footprint, divergence appeared in 22% of injection cases and in none of the autonomous or benign cases; where it appeared it was mechanism concealment, a description of a file write that omits the shell it spawns. A non-manipulated model mostly narrates honestly, and a manipulated one writes a faithful malicious description and drives the divergence to zero.

Reaction to a denial is real but points the other way. Given a blocked goal, a live agent adapted on 90% of steps, repeated on 10%, and stopped on none; it escalated from a plain redirection to `sudo`, to `su`, to `tee`, and to an alternative path, holding the objective fixed while varying the mechanism. The operational reading is that a denial elicits enumeration, not deterrence, which returns the weight to coverage: blocking one path routes the agent to the next.

9 Discussion

The result a defender should carry out of this study is a separation the marketing of syscall sandboxes tends to blur. A seccomp argument-inspection gate delivers a deterministic and auditable verdict, and that is worth having; it does not deliver certified containment, and presenting it as such misplaces the residual risk. The residual risk is coverage. It lives in the mechanisms the gate does not watch (Section 5), in the window between a check and the kernel’s re-read (Section 4), and in the imprecision of the argument match (Section 6). It does not live in the variance of the verdict, and it is not recovered by profiling the agent (Section 8).

Two design directions follow, and both are about closing coverage rather than tuning the agent. First, remove the second argument read on the allow path: resolve and copy the argument into memory the target cannot mutate, or let the supervisor perform the operation, so the value the supervisor checked is the value the kernel uses. Second, move from a blocklist to a default-deny allowlist over a race-free, path-resolving mechanism, so that coverage is the complement of an explicit permit set rather than the complement of an enumerated deny set. We do not report an implementation of either here; they are the shape a sound successor takes.

10 Limitations

The study is scoped and we state the scope rather than let a reviewer find it. The subjects are three local code models between 1.5 and 7 billion parameters, not frontier agents, and the autonomous destruction result rests on 45 attempting artifacts; both the rates and the specific mechanisms seen may shift with larger models and larger samples, and the emergent-destruction finding in particular is small- N . Every measurement was taken on a single `x86_64`

host. Interception was checked across three implementation languages but on one architecture; the aarch64 case is not established here, because faithful native seccomp semantics are not available under user-mode emulation, and a second-architecture host is the correct way to test it. The TOCTOU bypass is demonstrated as an existence result, which one successful race establishes; its per-attempt rate is scheduler-dependent and is not an external-validity claim. The behavioural-attribution experiments are forensic characterizations of non-adversarial actors and are reported as such; against an adversary that controls its own behaviour they attribute nothing, which is the point of Section 8.

11 Related work

Syscall-boundary containment for untrusted code predates LLM agents. Seccomp-BPF and its user-notification extension are the mechanism under test here; the deep-argument race we exercise is described in the kernel manual pages for user notification [9] and is not a new class of bug, only an instance of it in an agent-containment setting. Application kernels and sandboxes—gVisor [8], and the unprivileged, path-resolving Landlock LSM [7]—address the coverage and race problems this paper measures, and Landlock is the natural comparison for the file-operation policy of Section 6. Attribution of a process by its syscall behaviour goes back to the host-intrusion-detection line begun by Forrest et al., *A Sense of Self for Unix Processes* [1], which is the lineage our Section 8 sits in and declines to extend into a control. On the language side, that a chain-of-thought need not match behaviour is established—Turpin et al. [2] and Lanham et al. [3]—which is why the divergence signal cannot be a detector. The emerging agent-safety evaluations (ToolEmu [4]; AgentDojo [5]; AgentHarm [6]) measure agent harm at the tool or task level; this paper measures the containment control beneath them, at the syscall boundary. We position against these lines rather than beside them: prior sandboxing work supplies race-free mechanisms our tested gate lacks, and prior attribution work supplies the negative result our behavioural section confirms in the agent setting.

12 Conclusion

A deterministic syscall gate is a useful control and a poor certificate. Measured against an adversary rather than a cooperative target, the argument-inspection gate is bypassable on its allow path, blind to the mechanisms it does not enumerate, and imprecise where it matches paths by substring; and the agent it contains, when denied, looks for another way rather than stopping. The defensible claim is narrow and worth stating plainly: certify the verdict, scope the coverage, and assume the agent will route around any single blocked path. The open work is a supervisor with no second argument read and a default-deny allowlist over a path-resolving mechanism, and a repeat of these measurements against frontier models and a second architecture.

Reproducibility. All code, data, and per-experiment results are committed at `github.com/francose/llm-sandbox-cwe-research`. `experiments/agentsec/reproduce.sh` rebuilds the gate and the probes from source and re-checks the headline results.

References

- [1] S. Forrest, S. A. Hofmeyr, A. Somayaji, and T. A. Longstaff. A sense of self for Unix processes. In *Proc. 1996 IEEE Symposium on Security and Privacy*, pp. 120–128, 1996.
- [2] M. Turpin, J. Michael, E. Perez, and S. R. Bowman. Language models don’t always say what they think: unfaithful explanations in chain-of-thought prompting. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. arXiv:2305.04388.

- [3] T. Lanham et al. Measuring faithfulness in chain-of-thought reasoning. Anthropic, 2023. arXiv:2307.13702.
- [4] Y. Ruan et al. Identifying the risks of LM agents with an LM-emulated sandbox. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2309.15817.
- [5] E. Debenedetti et al. AgentDojo: a dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. In *NeurIPS Datasets and Benchmarks Track*, 2024. arXiv:2406.13352.
- [6] M. Andriushchenko et al. AgentHarm: a benchmark for measuring harmfulness of LLM agents. 2024. arXiv:2410.09024.
- [7] M. Salaün. Landlock: unprivileged access control. Linux kernel documentation, docs.kernel.org/userspace-api/landlock.html (merged in Linux 5.13; TCP network control in 6.7, ABI v4).
- [8] Google. gVisor: an application kernel for containers. <https://gvisor.dev>.
- [9] Linux man-pages project. `seccomp_unotify(2)`: seccomp user-space notification mechanism. https://man7.org/linux/man-pages/man2/seccomp_unotify.2.html.